Working with HashMap<K,V> programs :

```
package com.ravi.hash_map;

import java.util.HashMap;
import java.util.Iterator;
import java.util.Map;
import java.util.Map.Entry;

//BiConsumer  keySet()+loop  get()   getOrDefault()
//containsKey()  containsValue()    entrySet().iterator      entrySet()
public class HashMapDemo1
{
      public static void main(String[] args)
      {
        HashMap<Integer, String> map = new HashMap<>();
        map.put(1, "Vanilla");
        map.put(2, "Butterscotch");
        map.put(3, "Chocolate");
        map.put(4, "Cotton Candy");


        IO.println("By using toString() Method :");
        IO.println(map);

        IO.println("..................");


        IO.println("By using forEach(BiConsumer<K,V>)");
        map.forEach((key, value)-> IO.println("Key is :"+key+" value is :"+value));


        IO.println("..................");
        IO.println("Retrieving the value based on the key :");

        for(Integer key : map.keySet())  //[1, 2, 3, 4]
        {
             IO.println("Key "+key+" value is :"+map.get(key));
        }

        IO.println("..................");
        IO.println("get() & getOrDefault() Method :");
        Integer key = Integer.valueOf(IO.readln("Enter the key :"));
        String val = map.getOrDefault(key,"This key is not available");
        IO.println("Key "+key+" value is :"+val);


        IO.println("..................");
        IO.println("Searching the key ");

        key = Integer.valueOf(IO.readln("Enter the key :"));
        IO.println("Is key "+key+" available in the map :"+map.containsKey(key));


        IO.println("Searching the Value ");

        String value = IO.readln("Enter the Value :");
        IO.println("Is "+value+" available as a value :"+map.containsValue(value));


        IO.println("Size of the Map :"+map.size());

        IO.println("..................");

        IO.println("Iterating through iterator :");
        Iterator<Entry<Integer, String>> itr = map.entrySet().iterator();
        itr.forEachRemaining(IO::println);

        IO.println("Iterating through entrySet :");

        for(Map.Entry<Integer, String> entry : map.entrySet())
        {
          IO.println(entry.getKey()+" : "+entry.getValue());
        }
    }
}
```

```
package com.ravi.hash_map;

import java.util.HashMap;
import java.util.Map;

class LoginService
{
   private Map<String,String> users;

   public LoginService()
   {
     users = new HashMap<>();
   }

   public void register(String username, String password)
   {
     users.put(username, password);
     IO.println(username+" has registered in the application");
   }

   public boolean validte(String username, String password)
   {
     if(users.containsKey(username) && users.get(username).equals(password))
     {
           return true;
     }
     else
     {
           return false;
     }
   }
}

public class HashMapDemo2
{
   public static void main(String[] args)
   {
     LoginService service = new LoginService();
     service.register("Ravi", "Ravi999");
     if(service.validte("Ravi", "Ravi999"))
     {
       IO.println("Authenticated user");
     }
     else
     {
      System.err.println("Sorry!! Wrong username/password");
     }


   }
}
```

```
package com.ravi.hash_map;   // keySet()  values() + Loose coupling

import java.util.Collection;
import java.util.HashMap;
import java.util.Set;

record Student(Integer id, String name)
{

}


public class HashMapDemo3
{
  public static void main(String[] args)
  {
        HashMap<Integer,String> newmap1 = new HashMap<>();
        HashMap<Integer,String> newmap2 = new HashMap<>();

        newmap1.put(1, "OCPJP");
        newmap1.put(2, "is");
        newmap1.put(3, "best");

        newmap2.put(4, "exam");
        newmap2.put(5, "by Oracle");

        IO.println("putAll(Map<k,v> map");
        newmap2.putAll(newmap1);

        IO.println("Size of newmap2 :"+newmap2.size());

        newmap2.forEach((k,v)-> IO.println(k+" : "+v));

        IO.println("Printing all the keys :");
        Set<Integer> allTheKeys = newmap2.keySet();
        IO.println(allTheKeys);

        IO.println("Printing all the values :");
        Collection<String> values = newmap2.values();
        IO.println(values);

        IO.println("Loose Coupling");

        HashMap<Student, String> students = new HashMap<>();
        students.put(new Student(1, "Scott"), "Hyderabad");
        students.put(new Student(2, "Smith"), "Pune");
        students.put(new Student(3, "Alen"), "Indore");
        students.putIfAbsent(new Student(1, "Scott"), "S R Nagar");

        IO.println(students.put(new Student(2, "Smith"), "Ameerprt"));


        HashMap<Student, String> studentsMap = new HashMap<>(students);
        studentsMap.forEach((k,v)-> IO.println(k+" : "+v));

  }
}
```

```
package com.ravi.hash_map;

import java.util.HashMap;

record Employee(Integer empId, String empName)
{
}
public class HashMapDemo4
{
    public static void main(String[] args)
    {
        Employee e1 = new Employee(101,"Aryan");
        Employee e2 = new Employee(102,"Pooja");
        Employee e3 = new Employee(101,"Aryan");
        Employee e4 = e2;

        HashMap<Employee,String> hm = new HashMap<>();
        hm.put(e1,"Ameerpet");    //e1  Begumpet
        hm.put(e2,"S.R Nagar");   //e2  Panjagutta
        hm.put(e3,"Begumpet");
        hm.put(e4,"Panjagutta");
        hm.forEach((k,v)-> System.out.println(k+" : "+v));
    }
}
```

```
package com.ravi.hash_map;

import java.util.HashMap;
import java.util.Map;

public class HashMapDemo5
{
   public static void main(String[] args)
   {
      // Create a HashMap to store book titles and their availability (true = available, false = borrowed)

      HashMap<String, Boolean> library = new HashMap<>();

      library.put("Core Java", true);
      library.put("Advanced Java", true);
      library.put("HTML", false);
      library.put("JavaScript", true);

      IO.println("Initial Library Status :");

      for(Map.Entry<String,Boolean> status : library.entrySet())
      {
          IO.println(status.getKey()+" : "+status.getValue());
      }

     // Borrow a book
     String bookToBorrow = "Advanced Java";

     if(library.containsKey(bookToBorrow) && library.get(bookToBorrow))
     {
       library.put(bookToBorrow, false);
       IO.println(bookToBorrow+ " has borrowed successfully...");
     }
     else
     {
       IO.println(bookToBorrow+" book is not available for borrow");
     }

     String bookToReturn = "HTML";

     if(library.containsKey(bookToReturn) && !library.get(bookToReturn))
     {
       library.put(bookToReturn, true);

       IO.println(bookToReturn+" has returned by the borrower ");
     }
     else
     {
       IO.println(bookToReturn+" is not in the library");
     }


      library.forEach((k,v)-> System.out.println(k+" : "+v));


      // Check the availability of a book
      String bookToCheck = "JavaScript";

      if (library.containsKey(bookToCheck))
      {
        String availability = library.get(bookToCheck) ? "available" : "borrowed";
          System.out.println(bookToCheck + " Book is " + availability + ".");
      }
      else
      {
          System.out.println(bookToCheck + " is not in the library.");
      }


      //Display the final library status

      System.out.println("Final Library Status:");
      for (HashMap.Entry<String, Boolean> entry : library.entrySet())
      {
          String status = entry.getValue() ? "Available" : "Borrowed";
          System.out.println("Book: " + entry.getKey() + ", Status: " + status);
      }

   }
}
```